

LECTURE 13

Race Conditions and the Critical Section Problem

CSC323 – Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- What a race condition is, precisely, and why “it usually works” is not the same claim as “it is correct”
- A worked example at the assembly-instruction level, showing how two processes updating one shared variable can produce two different final answers depending only on timing
- A formal statement of the critical section problem and the three requirements any correct solution must satisfy: mutual exclusion, progress, and bounded waiting
- Why simply disabling interrupts during a critical section is a crude, incomplete fix
- Peterson's solution — a software-only algorithm for two processes — walked through and checked against all three requirements
- A subtle but critical caveat: why Peterson's solution can still fail on real modern hardware without an explicit memory barrier

Race Conditions

- A race condition occurs when two or more processes access and manipulate the same shared data concurrently, and the final outcome depends on the particular order in which their instructions happen to be scheduled.
- If every possible interleaving produces the same final result, there is no race — the outcome is determined, regardless of timing.
- A race condition exists precisely when at least two interleavings are possible and they disagree.

A WORKED EXAMPLE

The Shared Counter

```
counter++;    /* producer, after inserting an item */  
counter--;    /* consumer, after removing an item  */  
  
/* counter always holds the number of items  
   currently sitting in the bounded buffer */
```

Three Steps, Not One

```
; producer executing counter++
register1 = counter      ; load
register1 = register1 + 1 ; increment
counter   = register1    ; store

; consumer executing counter--
register2 = counter      ; load
register2 = register2 - 1 ; decrement
counter   = register2    ; store
```

Two Interleavings, Two Different Answers

Interleaving A — No Overlap

```
1. register1 = counter      → register1 = 5
2. register1 = register1 + 1 → register1 = 6
3. counter = register1      → counter = 6
4. register2 = counter      → register2 = 6
5. register2 = register2 - 1 → register2 = 5
6. counter = register2      → counter = 5
```

Final counter = 5 — correct

Interleaving B — Overlapped (Wrong)

```
1. register1 = counter      → register1 = 5
2. register2 = counter      → register2 = 5
3. register1 = register1 + 1 → register1 = 6
4. register2 = register2 - 1 → register2 = 4
5. counter = register1      → counter = 6
6. counter = register2      → counter = 4
```

Final counter = 4 — wrong; should be 5

Why Race Conditions Are Hard to Catch

A RACE CONDITION IS A TIMING BUG, NOT A LOGIC BUG

Reading the producer's code and the consumer's code separately reveals nothing wrong — each is individually correct. The bug only exists in the combination, under specific interleavings.

Those interleavings may be rare enough that a program runs correctly thousands of times in testing and then fails once in production under heavier load or on a different CPU.

This is precisely why race conditions are notorious: easy to introduce, and extremely hard to catch by inspection or by ordinary testing alone.

The Critical Section Problem

- The segment of code in which a process accesses data shared with at least one other process is called its critical section.
- The critical section problem is the problem of designing a protocol that processes can use to cooperate safely when each has a critical section to execute.
- No process may be allowed to run its critical section concurrently with another process's critical section, if those sections touch the same shared data.

The General Shape

```
do {  
    entry section  
        critical section  
    exit section  
        remainder section  
} while (true);
```

Four Sections, Four Jobs

- Entry section — code that requests permission to enter the critical section.
- Critical section — the actual access to shared data.
- Exit section — code that releases that permission.
- Remainder section — everything else the process does, where no sharing conflict exists.

Three Requirements for a Correct Solution

Mutual Exclusion

If a process is executing in its critical section, no other process may be executing in its own critical section for the same resource at the same time.

Progress

If no process is in its critical section and some process wants to enter, the decision of who enters next cannot be postponed indefinitely.

Bounded Waiting

There is a limit on how many times other processes may enter before a waiting process's request is granted — this rules out starvation.

Three Requirements, Three Failure Modes

THREE INDEPENDENT REQUIREMENTS, THREE INDEPENDENT FAILURE MODES

Violating mutual exclusion reproduces exactly the race condition traced earlier.

Violating progress can leave every process stuck in a decision no one is empowered to make — a kind of deadlock.

Violating bounded waiting allows one unlucky process to be passed over indefinitely even though the system keeps making progress — starvation.

Disabling Interrupts

```
disable_interrupts();  
    critical section  
enable_interrupts();
```

Why Disabling Interrupts Isn't Enough

- It does not work on a multiprocessor — disabling interrupts affects only the CPU core that executed the instruction. Any other core can schedule a different process straight into the same critical section.
- It hands every process a dangerous amount of power — a process that disables interrupts and then crashes, loops forever, or never re-enables them can freeze the entire machine, not just its own critical section.
- Even setting those aside, it is a blunt instrument: it blocks every interrupt, including timers and I/O completions unrelated to the resource being protected.

Two Shared Variables, One Loop

```
int turn;          /* whose turn, when both want in */
boolean flag[2];   /* flag[i] == true means Pi wants in */

/* code executed by process Pi (j is the other process) */
flag[i] = true;
turn = j;
while (flag[j] && turn == j) {
    /* busy wait */
}
/* critical section */
flag[i] = false;
/* remainder section */
```

Reading Peterson's Solution

- `flag[i] = true` says “I want to enter.”
- `turn = j` says “but I will politely let you go first, if you also want in right now.”
- The while condition says: keep waiting only while the other process also wants in AND it is genuinely the other process's turn.

Proof: Mutual Exclusion

- Suppose, for contradiction, that both P_i and P_j are inside their critical sections at the same time.
- For P_i to have passed its while loop, either $\text{flag}[j]$ was false or turn was not j — but P_j is also in its critical section, so $\text{flag}[j]$ must be true. That forces $\text{turn} == i$.
- By the symmetric argument, P_j getting through forces $\text{turn} == j$.
- turn is a single shared integer — it cannot equal both i and j at once. Contradiction. Mutual exclusion holds.

Proof: Progress

- A process sitting in its remainder section has flag set to false and therefore cannot block anyone — the other process's while condition sees `flag[j] = false` and proceeds.
- If only one process wants to enter, its own while condition is false immediately, so it proceeds without delay.
- If both want in, turn holds exactly one value, and whichever process that value favors proceeds — the decision is never postponed indefinitely.

Proof: Bounded Waiting

- Each time P_i attempts entry, it sets $turn = j$, explicitly deferring to the other process.
- If P_i exits, resets $flag[i]$, and later re-enters while P_j is still waiting, P_i 's new attempt again sets $turn = j$ — exactly the condition that lets the already-waiting P_j proceed next.
- A process can therefore be overtaken by the other at most once before its own entry is granted. No process can be skipped an unbounded number of times.

An Elegant Proof of Possibility

PETERSON'S SOLUTION NEEDS NO HARDWARE SUPPORT AT ALL

Every operation above — reading a boolean, writing a boolean, reading and writing an integer — is something any CPU could do with ordinary load/store instructions decades before specialized atomic instructions existed.

That is exactly what makes it an elegant proof of possibility: mutual exclusion does not strictly require special hardware.

The Catch: Reordering

PETERSON'S SOLUTION CAN FAIL ON REAL MODERN HARDWARE

The proof silently assumes that once P_i writes $\text{flag}[i] = \text{true}$ and $\text{turn} = j$, those writes are immediately visible to P_j before P_j evaluates its own while condition.

Modern compilers and CPUs do not guarantee that ordering by default — out-of-order execution and per-core store buffers can delay a write's visibility to other cores.

- If the write to $\text{flag}[i]$ (or turn) is reordered to happen after the read of $\text{flag}[j]$, both processes can observe the other's flag as still false and both enter the critical section at once — exactly as if no synchronization existed.

THE FIX

An Explicit Memory Barrier

```
flag[i] = true;
turn = j;
memory_barrier();      /* force the writes above
                        to be visible first */
while (flag[j] && turn == j) {
    /* busy wait */
}
```

A Proof of Concept, Not Production Code

- This is the main reason textbook presentations of Peterson's solution are best understood as a pedagogical proof of concept, not as code you would actually ship.
- Real systems never implement mutual exclusion with a hand-written Peterson loop; they build on hardware instructions specifically designed to carry the correct ordering and atomicity guarantees built in.
- That is exactly where Lecture 14 picks up: `test_and_set()`, `compare_and_swap()`, mutex locks, semaphores, and monitors.

Key Takeaways

- A race condition happens when the final result of concurrent access to shared data depends on the timing of instruction interleaving — the shared-counter example produced 5 under one interleaving and 4 under another.
- The critical section problem requires any solution to guarantee mutual exclusion, progress, and bounded waiting — three independent properties, each with its own failure mode.
- Disabling interrupts gives trivial mutual exclusion on a single CPU but fails outright on multiprocessors and hands any process the power to freeze the entire machine.
- Peterson's solution solves the two-process case with nothing but ordinary shared variables, and its three correctness properties can each be argued directly from the code.
- On real hardware, Peterson's solution needs an explicit memory barrier to stop the compiler or CPU from reordering its writes past its reads — without one, the algorithm's guarantees can silently collapse.